P2 learning report: everything so far

Author: Henry (P2, classification and evaluation) · **As of:** 2026-11-10 · **Notebook:** `notebooks/02_classification.ipynb`

This report walks through all of P2's work so far, in plain language: what was done, why, what came out, and what it means. Read it top to bottom once, then use the self-check questions at the end to test yourself before the debrief.

The earlier, more detailed note on the first model is `P2_logistic_regression_baseline.md`.

Contents

1. The big picture
2. The data
3. Model 1: logistic regression
4. Getting the data ready (preprocessing)
5. Testing fairly (cross-validation)
6. Scoring a model (ROC-AUC, PR-AUC, lift)
7. The shared features
8. The same model in PyTorch Lightning
9. Model 2: the MLP (neural network)
10. The validation study: how far can we trust the scores?
11. Scoreboard: every result in one place
12. What was done, in order
13. What's next
14. Self-check questions
15. Glossary

---

1. The big picture

The business problem. NkwaPay, a lender, gets loan applications and must decide: *approve or decline?* To decide well, it needs to know how likely each borrower is to **default**.

Default means the borrower **didn't pay the loan back**. In our data, `target = 1` means defaulted and `target = 0` means repaid. Only **1.83%** of loans defaulted, about **1 in 55**.

P2's job has two halves:

	Classification	Evaluation
Question	"Will this loan default?"	"How far can we trust those predictions?"
Done so far	Logistic regression, MLP	ROC-AUC, PR-AUC, fair cross-validation, validation study
Still to do		Calibration, cost-based approve/decline cutoff, confidence intervals

A model gives a **probability of default** for each loan (for example "12%"). Evaluation checks whether those probabilities are good enough to base lending decisions on.

---

2. The data

Which file we learn from

File	Has the answer (`target`)?	Used for
`Train.csv` (68,654 loans)	**Yes**	Learning **and** checking (by hiding part of it)
`Test.csv` (18,594 loans)	**No**	Only the final competition submission

A model can only learn from examples where it can check its guesses. That's why everything is built on `Train.csv`.

Which columns are clues, and which are not

Column(s)	Used?	Why
Loan amount, duration, lender's share, new/repeat borrower, lender, loan type	✅ Clues (features)	Known when the loan is approved
`Total_Amount_to_Repay`, `Lender_portion_to_be_repaid`	❌ **Leaks**	Each alone predicts default almost perfectly (AUC 0.98), most likely because late penalties are added *after* a default. Using them is like reading the last page of the novel.
`ID`, `customer_id`, `tbl_loan_id`	❌	Labels for rows, people and loans, not information about risk
`due_date`, `Amount_Funded_By_Lender`	❌	Can be calculated from other columns, so they add nothing
`country_id`	❌ as a clue, ✅ for checking	Every training loan is from **Kenya**. A column that never changes can't teach anything. Kept for comparing Kenya vs Ghana later.

---

3. Model 1: logistic regression

How it thinks: a points system

Fact about the loan	Points (made-up example)
New borrower	+3 (risky)
Repeat borrower	−1 (safer)
Large loan	+0.5

1. Add up the points to get a **risk score**.
2. Pass the score through the **sigmoid**, an S-shaped curve that turns any number into a probability between 0 and 1.

How it learns: training

The points (called **weights**) aren't chosen by us. The model learns them:

> guess each loan's probability → compare with what really happened → adjust the weights to be less wrong → repeat

A big positive weight pushes towards "default"; a big negative weight pushes towards "repays".

The rare-default problem

Since only 1 in 55 loans defaults, a model that says "nobody defaults" is **98% accurate and completely useless**. To prevent this, we make each default count about **54× more** during training (`class_weight="balanced"` in scikit-learn, `pos_weight≈54` in Lightning). 54 ≈ (non-defaults ÷ defaults).

---

4. Getting the data ready (preprocessing)

Step	What it does	Why
Log of money amounts	Shrinks huge values (a few loans are millions)	So the giants don't dominate
Scaling	Puts every number on the same scale (mean 0, spread 1)	So no column matters more just because its numbers are bigger
One-hot encoding	Turns "Lender A" into its own 0/1 column	Models only understand numbers
`handle_unknown="ignore"`	Unseen categories (like new Ghana lenders) become all zeros	So the model doesn't crash on them
Imputation (filling blanks)	Fills empty values with a typical value, plus a "this was blank" column	First-time borrowers have no history; having no history is itself a clue

The golden rule: the cleaning steps are learned from the **training rows only**. If the scaler saw the test rows, information would leak and scores would look better than they are. A scikit-learn **Pipeline** enforces this automatically.

---

5. Testing fairly (cross-validation)

We hide part of `Train.csv` from the model and test on it:

1. Split the loans into **5 groups (folds)**.
2. Train on 4, test on the 5th, which the model has never seen.
3. Repeat so each group is the test once, then average.

GroupKFold by customer: all of one customer's loans stay in the same fold. Otherwise the model could be tested on a borrower it already studied, like seeing the exam questions in advance.

---

6. Scoring a model (ROC-AUC, PR-AUC, lift)

Both scores grade how well the model **ranks** loans from riskiest to safest.

ROC-AUC: "Does it rank defaulters above repayers?"

Pick one random defaulter and one random repayer. ROC-AUC is the chance the model gives the defaulter the higher risk. - 1.0 = perfect · **0.5 = coin flip**

PR-AUC: "When it flags someone, is it right, and does it catch most defaulters?"

It balances: - **Precision:** of the loans we flagged, how many really defaulted? - **Recall:** of all the real defaulters, how many did we flag?

Random guessing scores the default rate, not 0.5. With 1.8% defaults, random gets 0.018.

Lift = PR-AUC ÷ default rate

"How many times better than random?" We need this when comparing test sets with **different default rates** (see Section 10).

Why PR-AUC is our main score: with rare defaults, ROC-AUC can look flattering. Wrongly flagging 5% of ~67,000 good borrowers sounds small but is about 3,350 people. PR-AUC focuses on the rare defaulters.

---

7. The shared features

What and why

Perfect (P1) built **one official version of the data** for the whole team (`src/features.py` → `data/processed/features.parquet`). It's **the same loans from `Train.csv`, with the same answers, plus extra clues**, and the **test folds already decided** for every loan.

Everyone uses it, so **everyone sits the same exam** and scores are comparable.

The extra clues

Clues	Plain meaning
`n_past_loans`, `n_past_defaults`, `past_default_rate`, `days_since_last_loan`, `days_since_last_default`	Borrower's history: how many loans before, how often they failed to repay, how recently
`disb_month`, `disb_dow`, `disb_dom`	Month, weekday, day of month the loan was given
`macro_inflation`, `macro_fx`, …	The country's economy in the previous year
`log_amount_usd`	Amount in US dollars, so shillings and cedis compare

Time-safe: a past default only counts if its due date was before the new loan was given, because before that nobody knew the outcome. The model never peeks into the future.

What changed in our results

Clues the model sees	ROC-AUC	PR-AUC
Original 7 columns	0.79	0.19
+ borrower history	**0.84**	0.19
All shared features (official)	**0.845**	**0.170**

Lesson: history helps overall ranking, but extra calendar and economy columns slightly *lower* PR-AUC. Logistic regression gives every column a weight, so weak columns add noise. **More columns ≠ better model.**

---

8. The same model in PyTorch Lightning

Why build it twice?

scikit-learn hides everything in one line. In Lightning we write each piece, which shows we understand it, proves our training loop is correct, and sets up the MLP.

Piece	What it does
`nn.Linear`	One weight per column, plus a bias: the points system
`BCEWithLogitsLoss`	Measures how wrong the guesses are (applies the sigmoid itself)
`pos_weight ≈ 54`	Each default counts about 54× more
Adam optimizer	Nudges the weights to be less wrong
`weight_decay`	Small penalty that keeps weights from growing huge
Batch (1,024 loans)	Learn from small groups at a time
Epoch	One full pass through the training loans
Early stopping	Stop when the score on held-out loans stops improving

Careful detail: early stopping watches **10% of the training customers**, never the test fold, so the test stays completely unseen.

Result

scikit-learn and Lightning **both scored 0.845 / 0.170**, so the hand-built loop is correct. Their predictions were 97% correlated. Individual weights matched only about 80%, because some columns carry the same information and the credit can be split between them differently.

Practical: use the project's `.venv` kernel; on a Mac use `accelerator="cpu"` and `float32`.

---

9. Model 2: the MLP (neural network)

MLP = Multi-Layer Perceptron. A perceptron is one artificial neuron, and logistic regression is basically one. An MLP stacks many of them in layers:

```
clues → [64 neurons] → [32 neurons] → [1 neuron] → probability
          hidden         hidden        output
```

Why it can do better: logistic regression only *adds up* separate clues. Each hidden neuron learns a **combination** of clues (for example "new borrower + small loan + lender X"), and the next layer combines those.

New ingredient	What it does	Why
ReLU	Keeps positives, turns negatives into 0	Without it, layers collapse back into one logistic regression
Dropout (20%)	Randomly switches off 20% of neurons on each training batch	Prevents memorising the training loans (overfitting)
`model.eval()`	Switches dropout off when predicting	You want every neuron working for real predictions

Result

Model	ROC-AUC	PR-AUC
Logistic regression	0.845	0.170
MLP	**0.865**	**0.215**

The MLP was ahead **in all 5 folds**. Smaller and bigger networks scored about the same. **Caution:** "ahead in all folds" is strong evidence, but the formal proof comes with confidence intervals (#20). And Section 10 shows a big catch.

---

10. The validation study: how far can we trust the scores?

The question

How we split the data for testing changes the score. Which score is **honest**?

Split	How	Question it answers
Random (`fold_strat`)	Shuffle loans into 5 folds	The naive way
By customer (`fold_group`)	Each customer in one fold only	What we've used so far
By time (`temporal`)	Train on loans **before 2023**, test on **2023 onwards**	"Can we predict the future?"

Results

Model	Split	ROC-AUC	PR-AUC	Default rate in test	Lift
LogReg	Random	0.851	0.185	1.8%	10.1×
LogReg	By customer	0.845	0.170	1.8%	9.3×
LogReg	**By time**	**0.620**	0.128	**9.5%**	**1.3×**
MLP	Random	0.869	0.220	1.8%	12.0×
MLP	By customer	0.867	0.215	1.8%	11.8×
MLP	**By time**	**0.480**	0.091	**9.5%**	**0.95×**

Lesson 1: random vs by-customer, small inflation

In a random split, **94%** of test customers also appear in training, yet the score rises only slightly (+0.006 ROC-AUC). Mixing customers barely flatters the model here.

Lesson 2: by time, the models fall apart

- Logistic regression drops to **0.62**: it still knows something, but little.
- The MLP drops to **0.48**, **worse than a coin flip**. The flexible model learned 2022's patterns so closely that they stopped working later. **The model that won in Section 9 loses badly here.**

Lesson 3: the later loans are a different population

	Before 2023 (training)	2023 onwards (test)
Main lender's share	**97%**	49%
Average loan length	7.8 days	**19.8 days**
Average past loans per borrower	15.7	**5.0**
New borrowers	0.6%	3.8%
Default rate	1.3%	**9.5%**

Removing calendar, economy or lender columns **didn't help** (still 0.62). So the problem isn't one bad feature: **the lending business itself changed.** This is called **distribution shift**, and it's exactly what Part

C (Kenya → Ghana) is about.

Why we use "lift" here

Random guessing scores the default rate: 1.8% in the folds, **9.5%** in the time test. So a raw PR-AUC of 0.128 in the time test is weaker than it looks. Lift (PR-AUC ÷ default rate) puts them on the same footing.

What we can honestly promise (decision D-7, to agree with the team)

> **Compare models with GroupKFold** (stable, five folds, little inflation). **Report the time-based score as the honest forecast** for future loans, and state clearly that performance drops sharply when lenders, loan terms and borrowers change. Prefer the simpler, more robust model when the future may differ from the past.

---

11. Scoreboard: every result in one place

#	Model	Data	Split	ROC-AUC	PR-AUC
1	LogReg (scikit-learn)	7 raw columns	By customer	0.792	0.195
2	LogReg (scikit-learn)	Shared features	By customer	0.845	0.170
3	LogReg (Lightning)	Shared features	By customer	0.845	0.170
4	MLP (Lightning)	Shared features	By customer	0.865	0.215
5	LogReg	Shared features	Random	0.851	0.185
6	MLP	Shared features	Random	0.869	0.220
7	LogReg	Shared features	**By time**	**0.620**	0.128 (lift 1.3×)
8	MLP	Shared features	**By time**	**0.480**	0.091 (lift 0.95×)

---

12. What was done, in order

Step	What	Where
1	Logistic regression on 7 raw columns	PR #40
2	Plain-language explainer	`docs/learning/P2_logistic_regression_baseline.md`
3	Switched to the shared features and folds	PR #41
4	Same model in PyTorch Lightning; matches scikit-learn	PR #41
5	WORKLOG, PROGRESS, AI-use log entries	PR #41
6	README turned into a task checklist, grouped by person	PRs #43, #44
7	MLP; **issue #16 closed**	commit `7175a40` on `main`
8	Validation study: random vs by customer vs by time	`02_classification.ipynb`, branch `p2/validation-study`

13. What's next

Issue	Task	Status
#14	Validation study	🟨 Study done; write conclusions (cell R) and agree D-7 with the team
#10 + #11	Define the two tasks; cost matrix in cedis/shillings	⬜
#19	Calibration and cost-based approve/decline cutoff	⬜
#20	Bootstrap confidence intervals (is the MLP really better?)	⬜
#13	`src/train.py`, `src/evaluate.py`, `src/datamodule.py`	🟨 Training loop done
#6	Target EDA by month, lender, loan type	🟨 By year done
#32	Logistic regression derivation	⬜

14. Self-check questions

Try to answer out loud before opening each answer. At the debrief you must explain this without notes.

1. What does "default" mean, and how rare is it here?

Answer. The borrower didn't repay. 1.83% of loans, about 1 in 55.

2. Why do we train on Train.csv and not Test.csv?

Answer. Only Train.csv has the answer column. A model can't learn without being able to check its guesses. Test.csv is only for the final submission.

3. Why were Total_Amount_to_Repay and Lender_portion_to_be_repaid removed?

Answer. They're leaks. Each alone predicts default with AUC 0.98, most likely because penalties are added after a default, so they contain the answer. They wouldn't be known when deciding on a loan.

4. Why is "approve everyone" 98% accurate, and why is that useless?

Answer. 98% of loans are repaid, so always predicting "repays" is right 98% of the time but catches zero defaulters. That's why we use PR-AUC and weight defaults more.

5. What does a PR-AUC of 0.17 mean, and what does random guessing score?

Answer. It measures how well the model finds defaulters with few false alarms. Random guessing scores the default rate (0.018), so 0.17 is about 9× better than random.

6. Why must the scaler be fitted on training rows only?

Answer. Otherwise the test rows' averages leak into training and the score looks better than it really is.

7. Why GroupKFold by customer?

Answer. So the model is never tested on a borrower it already studied, which would inflate the score.

8. Why did we build logistic regression twice?

Answer. To prove the hand-built Lightning training loop is correct (both gave 0.845 / 0.170) and to set up the MLP.

9. What can an MLP learn that logistic regression can't?

Answer. Combinations of clues. Logistic regression only adds up separate effects; hidden layers with ReLU learn interactions.

10. What do ReLU and dropout do?

Answer. ReLU lets layers bend (without it, the layers collapse into one linear model). Dropout randomly switches off neurons during training to prevent memorising.

11. Why is the time-based score so much lower than the random one?

Answer. Loans from 2023 onwards come from a different population: a different lender mix, loan lengths 2.5× longer, borrowers with less history, and a default rate of 9.5% instead of 1.3%. Patterns learned from 2022 don't carry over.

12. The MLP beat logistic regression in the folds but lost in the time test. Why?

Answer. Its flexibility let it fit 2022's specific patterns closely. When the business changed, those patterns broke, and it scored worse than random (0.48). The simpler model degraded less (0.62).

13. What can we honestly promise the committee?

Answer. The model ranks loans well (≈0.85 ROC-AUC) as long as new loans resemble past ones. Under the lender and product changes seen in 2023, performance drops sharply (≈0.62 or worse), so the model must be monitored and retrained when the business changes.

---

15. Glossary

Word	Meaning
Default	Borrower didn't repay (`target = 1`)
Feature / clue	A column the model uses
Label / target	The answer we predict
Leak	A column that secretly contains the answer
Weight	The "points" a model learns per feature
Sigmoid	S-curve that turns a score into a probability
Training	Learning the weights from past loans
Class imbalance	One outcome is rare
Preprocessing	Cleaning and converting clues into numbers
One-hot encoding	A category becomes 0/1 columns
Imputation	Filling blank values
Pipeline	Chains cleaning and model so cleaning is learned from training rows only
Cross-validation	Testing on held-out folds, in rotation
GroupKFold	Cross-validation keeping each customer's loans together
Temporal split	Train on the past, test on the future
ROC-AUC	Chance a defaulter is ranked above a repayer (0.5 = random)
PR-AUC	Precision vs recall summary (random = default rate)
Precision	Of loans flagged, share that really defaulted
Recall	Of real defaulters, share that were flagged
Lift	PR-AUC ÷ default rate: times better than random
Shared features	The team's one official data file
Loss	Number measuring how wrong the guesses are
Batch	Small group of loans learned from at once
Epoch	One full pass through the training loans
Early stopping	Stop when held-out performance stops improving
Overfitting	Memorising training data instead of learning general patterns
MLP	Multi-layer perceptron: a neural network of stacked neuron layers
Neuron / perceptron	Weighted sum of inputs, like a mini logistic regression
Hidden layer	Middle layer of an MLP

Word	Meaning
ReLU	Keeps positives, zeroes negatives; lets networks bend
Dropout	Randomly switching off neurons during training
Distribution shift	New data differs from training data, so models degrade